



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: _12_____

Team No.: __24_____

Project Title: Linux File System Explorer Using System Calls

Team Members

Roll Number	Student Name	Signature
2520090135	M. Shiva	
2520090115	N.Sainath Reddy	

1. Abstract

(Summarize the project idea, problem background, relevant Operating Systems/System Programming concepts, proposed Linux-based solution, and expected outcome. The abstract should not exceed 250 words.)

Linux File System Explorer Using System Calls is a Linux-based system programming project that provides a command-line interface for exploring and performing basic file-system operations using low-level Linux/POSIX system calls. The project addresses the need to understand how user applications interact with the Linux kernel instead of relying only on graphical file managers.

The system will demonstrate important Operating Systems concepts such as file descriptors, file and directory management, file permissions, metadata, and kernel-level resource access. It will use C programming on Ubuntu/Linux with APIs and system calls such as open(), read(), write(), close(), lseek(), stat(), mkdir(), opendir(), readdir(), and related POSIX mechanisms. Users will be able to navigate directories, display file information, create files/directories, read and write files, and perform basic file-management operations. The expected outcome is a working command-line file-system explorer that strengthens practical understanding of Linux system calls, file handling, directory structures, and Operating Systems concepts.

2. Problem Statement

(Clearly describe the problem that the project intends to address and its relevance to Operating Systems and Systems Programming.)

Many file-management applications hide the low-level operations performed by the operating system, making it difficult for students to understand how files, directories, metadata, permissions, and file descriptors are handled by Linux. This project addresses the problem by developing a simple command-line File System Explorer that directly uses Linux/POSIX system calls. It will allow users to perform common file and directory operations while demonstrating how a user program communicates with the Linux kernel. The project is relevant to Operating Systems and Systems Programming because it provides practical implementation of file-system interfaces, system calls, file descriptors, directory traversal, and resource management.

3. Objectives

1. Implement a Linux command-line file-system explorer using C and POSIX/Linux system calls.
2. Provide basic operations such as directory navigation, file creation, reading, writing, and listing.
3. Display file metadata such as size, type, permissions, and timestamps using appropriate APIs.
4. Demonstrate practical concepts of file descriptors, system calls, directory handling, and kernel interaction.

4. Proposed Methodology

The project will be implemented in C on Ubuntu/Linux as a menu-driven command-line application. The program will accept user commands and map them to suitable Linux/POSIX APIs. Directory-related operations will use directory APIs such as `opendir()` and `readdir()`, while file operations will use system calls such as `open()`, `read()`, `write()`, `close()`, and `lseek()`. File information will be obtained using `stat()` and related metadata structures.

The application flow will be: start the program → display the current directory/menu → accept a user operation → validate the input/path → invoke the required Linux API or system call → display the result or error message → return to the menu until the user exits. Error handling will be included using return values and appropriate error-reporting mechanisms. The program will be compiled using GCC and tested through the Linux terminal.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
<code>open()</code> , <code>close()</code>	Open a file and release its file descriptor.
<code>read()</code> , <code>write()</code>	Read data from and write data to files.
<code>lseek()</code>	Move the file offset for random file access.
<code>stat()</code>	Retrieve file metadata such as size, type, permissions, and timestamps.
<code>opendir()</code> , <code>readdir()</code>	Open a directory and traverse its entries.
<code>mkdir()</code> , <code>rmdir()</code>	Create and remove directories.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090135	M.Shiva	Design the command-line interface and overall program flow. Implement file metadata, permissions, error handling, and testing. Prepare documentation, GitHub repository, test cases, and final presentation
2520090115	N.Sainath Reddy	Implement file operations using <code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code> , and <code>lseek()</code> . Implement directory traversal, creation, and removal using directory/POSIX APIs.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution environment.
C	Primary programming language for system-call implementation.
GCC	Compile and build the C program.
VS Code	Source-code editing and project development.
Linux Terminal	Run, test, and demonstrate the application.

8. Expected Outcome

The completed application is expected to provide a functional command-line File System Explorer through which users can list directory contents, navigate directories, create files and directories, read and write file contents, inspect file metadata, and perform selected basic file-management operations.

The project will produce clear command-line outputs and error messages while demonstrating the use of Linux system calls, POSIX APIs, file descriptors, directory structures, permissions, and file metadata. It will provide practical experience in connecting C programs with Linux kernel services.

9. GitHub Repository

GitHub Repository Link: _____

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: M. Raghupathi

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____